# Understanding Dockerfile Syntax

In this train, we'll delve into the basic syntax and commands used in a Dockerfile, including FROM, RUN, CMD, COPY, and EXPOSE, to help you understand how Docker images are built.

## Learning Objectives

By the end of this train you should be able to:

- Recognize and utilize the basic syntax and commands of a Dockerfile, including FROM, RUN, CMD, COPY, and EXPOSE.

## Essential Dockerfile Commands

The following five commands are the workhorses of most Dockerfiles, forming the foundation for building effective Docker images.

### FROM

The FROM instruction sets the base image for subsequent instructions. It's the starting point of your Docker image.

**Example**: `FROM python:3.8-slim`

Here we use a slim version of Python 3.8, as our base image.

> **Note:**
> Choosing the right base image is crucial. It should be as minimal as

possible while providing the necessary foundation for your application.

## RUN ✖

The RUN instruction executes commands within the Docker image during the build process. It's used to install software, create directories, and perform other setup tasks.

**Example**: `RUN pip install --no-cache-dir -r requirements.txt`

Here we install Python dependencies

> **Note:**
> It is best practice to combine multiple commands into a single RUN instruction using && to minimize the number of image layers.

## CMD ✖

The CMD instruction specifies the default command that will be executed when a container is run from the image.

**Example**: `CMD ["python", "app.py"]`

Here run a python script.

## COPY ✖

The COPY instruction copies files or directories from the host machine (the build context) into the Docker image.

**Example**: `COPY app.py /app/`

Here we copy app.py to the app directory.

**Note:**
It is best practice to copy only the necessary files. Try and avoid copying unnecessary data to keep the image size small.

**EXPOSE** ✖

The EXPOSE instruction informs Docker that the container will listen on a specific network port at runtime.

**Example**: `EXPOSE 80`

Here we indicate that the container will listen on port 80

## Conclusion

These five commands form the core of most Dockerfiles. By understanding their purpose and syntax, you can build effective Docker images for your applications and data pipelines.

**Key Takeaways** ✖

- **Takeaway 1:** FROM - Specify the base image of your Docker image.
- **Takeaway 2:** `RUN` - Run commands inside the image while it's being built (Setting things up).
- **Takeaway 3:** `COPY` - Copy files into the Docker image filesystem.
- **Takeaway 4:** `CMD` - Sets a command to be run when a container is created.
- **Takeaway 5:** Specifcy which network ports to listen.